

EXAMEN TITULO

Este código implementa un sistema básico de biblioteca en Python. Incluye clases para manejar usuarios, libros y préstamos. Permite buscar libros por autor, registrar préstamos y devoluciones, y gestionar penalidades para los usuarios

```
from datetime import date

class Usuario:
    def __init__(self, nombre: str, id_usuario: int):
        self._nombre = nombre
        self.__id_usuario = id_usuario

    def get_nombre(self) -> str:
        return self._nombre

    def set_nombre(self, nombre: str):
        self._nombre = nombre

    def get_id_usuario(self) -> int:
        return self.__id_usuario

    def set_id_usuario(self, id_usuario: int):
        self.__id_usuario = id_usuario

    def buscar_autor(self, autor: str, catalogo: list) -> list:
        return [libro for libro in catalogo if libro.get_autor().lower() == autor.lower()]

class UsuarioBiblioteca(Usuario):
    def __init__(self, nombre: str, id_usuario: int, penalidades: float = 0.0):
        super().__init__(nombre, id_usuario) # Llamada al constructor de la clase base
        self.penalidades = penalidades

    def agregar_penalidad(self, monto: float):
        self.penalidades += monto

class Libro:
    def __init__(self, titulo: str, autor: str, isbn: str, estado: str = "disponible"):
        self.__titulo = titulo
```

```

        self.__autor = autor
        self.__isbn = isbn
        self.__estado = estado

    def get_titulo(self) -> str:
        return self.__titulo

    def get_autor(self) -> str:
        return self.__autor

    def get_estado(self) -> str:
        return self.__estado

    def set_estado(self, estado: str):
        self.__estado = estado

class Prestamo:
    def __init__(self, usuario: Usuario, libro: Libro):
        self.usuario = usuario
        self.libro = libro
        self.fecha_prestamo = date.today()
        self.fecha_devolucion = None

    def realizar_prestamo(self):

        if self.libro.get_estado() == "disponible":
            self.libro.set_estado("prestado")
            print(f"Préstamo realizado: {self.libro.get_titulo()} para {self.usuario.get_nombre()}")
        else:
            raise Exception(f"El libro '{self.libro.get_titulo()}' no está disponible para préstamo.")

    def devolver_libro(self):
        try:
            self.libro.set_estado("disponible")
            self.fecha_devolucion = date.today()
            print(f"Libro '{self.libro.get_titulo()}' devuelto por {self.usuario.get_nombre()}")
        except Exception as e:
            print(f"Error al devolver el libro: {e}")

catalogo = [
    Libro("Libro A", "Autor 1", "123"),
    Libro("Libro B", "Autor 2", "456"),

```

```
        Libro("Libro C", "Autor 1", "789"),
    ]

usuario = UsuarioBiblioteca("Juan Pérez", 1)
libro = catalogo[0]

try:
    prestamo = Prestamo(usuario, libro)
    prestamo.realizar_prestamo()
except Exception as e:
    print(f"Error en el proceso de préstamo: {e}")

print("\nLibros del Autor 1:")
for libro in usuario.buscar_autor("Autor 1", catalogo):
    print(f"- {libro.get_titulo()} ({libro.get_estado()})")

try:
    prestamo.devolver_libro()
except Exception as e:
    print(f"Error en el proceso de devolución: {e}")
```

Pantalla de Salida:

```
Préstamo realizado: Libro A para Juan Pérez

Libros del Autor 1:
- Libro A (prestado)
- Libro C (disponible)
Libro 'Libro A' devuelto por Juan Pérez.
```

Diagrama de clases y casos de uso

